BookMyShow – High-Concurrency Booking Engine

P1 – Entities and Attributes

1. users
user_id, full_name, email, phone, created_at

2. movies
movie_id, title, duration_minutes, language, certificate, release_date, created_at

3. theatres
theatre_id, theatre_name, address, city, state, postal_code, created_at

4. screens
screen_id, theatre_id, screen_name, total_seats

5. seats
seat_id, screen_id, row_label, seat_number, seat_type

6. shows
show_id, screen_id, movie_id, show_date, start_time, end_time, base_price, status

7. show_seats
show_seat_id, show_id, seat_id, price, status, hold_token, hold_expires_at, version

8. bookings
booking_id, booking_reference, user_id, show_id, status, total_amount, hold_expires_at, created_at

9. booking_seats
booking_id, show_seat_id, unit_price

10. payments
payment_id, booking_id, provider, provider_payment_id, amount, currency, status

11. payment_webhooks
webhook_id, provider, provider_event_id, event_type, provider_payment_id, payload, processing_status

Table Structure / SQL

```
-- BookMyShow – High Concurrency Booking Engine
-- MySQL 8.x

CREATE DATABASE IF NOT EXISTS bookmyshow_db;
USE bookmyshow_db;

CREATE TABLE users (
    user_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL UNIQUE,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE movies (
    movie_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    duration_minutes INT NOT NULL,
    language VARCHAR(50) NOT NULL,
    certificate VARCHAR(10),
    release_date DATE,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE theatres (
    theatre_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    theatre_name VARCHAR(200) NOT NULL,
    address VARCHAR(255) NOT NULL,
    city VARCHAR(100) NOT NULL,
    state VARCHAR(100) NOT NULL,
    postal_code VARCHAR(20) NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_theatre_city (city)
```

```
) ENGINE=InnoDB;

CREATE TABLE screens (
    screen_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    theatre_id BIGINT UNSIGNED NOT NULL,
    screen_name VARCHAR(100) NOT NULL,
    total_seats INT NOT NULL,
    UNIQUE KEY uk_theatre_screen (theatre_id, screen_name),
    FOREIGN KEY (theatre_id) REFERENCES theatres(theatre_id)
) ENGINE=InnoDB;

CREATE TABLE seats (
    seat_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    screen_id BIGINT UNSIGNED NOT NULL,
    row_label VARCHAR(10) NOT NULL,
    seat_number INT NOT NULL,
    seat_type ENUM('REGULAR','PREMIUM','RECLINER') NOT NULL DEFAULT 'REGULAR',
    UNIQUE KEY uk_seat_position (screen_id, row_label, seat_number),
    FOREIGN KEY (screen_id) REFERENCES screens(screen_id)
) ENGINE=InnoDB;

CREATE TABLE shows (
    show_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    screen_id BIGINT UNSIGNED NOT NULL,
    movie_id BIGINT UNSIGNED NOT NULL,
    show_date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    base_price DECIMAL(10,2) NOT NULL,
    status ENUM('SCHEDULED','CANCELLED','COMPLETED')
        NOT NULL DEFAULT 'SCHEDULED',
    UNIQUE KEY uk_screen_show (screen_id, show_date, start_time),
    INDEX idx_show_date_screen (show_date, screen_id, start_time),
    FOREIGN KEY (screen_id) REFERENCES screens(screen_id),
    FOREIGN KEY (movie_id) REFERENCES movies(movie_id)
) ENGINE=InnoDB;

CREATE TABLE show_seats (
    show_seat_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    show_id BIGINT UNSIGNED NOT NULL,
    seat_id BIGINT UNSIGNED NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    status ENUM('AVAILABLE','HELD','BOOKED') NOT NULL DEFAULT 'AVAILABLE',
    hold_token CHAR(36),
    hold_expires_at DATETIME,
    version INT NOT NULL DEFAULT 0,
    UNIQUE KEY uk_show_seat (show_id, seat_id),
    INDEX idx_show_seat_status (show_id, status),
    INDEX idx_hold_expiry (status, hold_expires_at),
    FOREIGN KEY (show_id) REFERENCES shows(show_id),
    FOREIGN KEY (seat_id) REFERENCES seats(seat_id)
) ENGINE=InnoDB;

CREATE TABLE bookings (
    booking_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    booking_reference VARCHAR(20) NOT NULL UNIQUE,
    user_id BIGINT UNSIGNED NOT NULL,
    show_id BIGINT UNSIGNED NOT NULL,
    status ENUM('PENDING','CONFIRMED','CANCELLED','EXPIRED')
        NOT NULL DEFAULT 'PENDING',
    total_amount DECIMAL(12,2) NOT NULL,
    hold_expires_at DATETIME,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX idx_booking_expiry (status, hold_expires_at),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (show_id) REFERENCES shows(show_id)
) ENGINE=InnoDB;

CREATE TABLE booking_seats (
    booking_id BIGINT UNSIGNED NOT NULL,
    show_seat_id BIGINT UNSIGNED NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (booking_id, show_seat_id),
    UNIQUE KEY uk_booked_show_seat (show_seat_id),
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),
    FOREIGN KEY (show_seat_id) REFERENCES show_seats(show_seat_id)
) ENGINE=InnoDB;

CREATE TABLE payments (
    payment_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    booking_id BIGINT UNSIGNED NOT NULL,
    provider VARCHAR(50) NOT NULL,
    provider_payment_id VARCHAR(150) NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    currency CHAR(3) NOT NULL DEFAULT 'INR',
    status ENUM('INITIATED','AUTHORIZED','CAPTURED','FAILED','REFUNDED')
        NOT NULL DEFAULT 'INITIATED',
    UNIQUE KEY uk_provider_payment (provider, provider_payment_id),
    UNIQUE KEY uk_payment_booking (booking_id),
```

```
    FOREIGN KEY (booking_id) REFERENCES bookings(booking_id)
) ENGINE=InnoDB;

CREATE TABLE payment_webhooks (
    webhook_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    provider VARCHAR(50) NOT NULL,
    provider_event_id VARCHAR(150) NOT NULL,
    event_type VARCHAR(100) NOT NULL,
    provider_payment_id VARCHAR(150),
    payload JSON NOT NULL,
    processing_status ENUM('RECEIVED','PROCESSED','FAILED')
        NOT NULL DEFAULT 'RECEIVED',
    received_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    processed_at DATETIME,
    UNIQUE KEY uk_provider_event (provider, provider_event_id),
    INDEX idx_webhook_payment (provider, provider_payment_id)
) ENGINE=InnoDB;
```

Sample Entries

```
INSERT INTO users (full_name, email, phone) VALUES
('Rahul Sharma', 'rahul@example.com', '9000000001'),
('Priya Kumar', 'priya@example.com', '9000000002');

INSERT INTO movies
(title, duration_minutes, language, certificate, release_date)
VALUES
('The Last Mission', 145, 'English', 'UA', '2026-09-18'),
('Kannada Express', 132, 'Kannada', 'U', '2026-09-20');

INSERT INTO theatres
(theatre_name, address, city, state, postal_code)
VALUES
('PVR Orion Mall', 'Dr Rajkumar Road', 'Bengaluru',
 'Karnataka', '560055');

INSERT INTO screens (theatre_id, screen_name, total_seats) VALUES
(1, 'Audi 1', 6);

INSERT INTO seats
(screen_id, row_label, seat_number, seat_type)
VALUES
(1, 'A', 1, 'REGULAR'),
(1, 'A', 2, 'REGULAR'),
(1, 'A', 3, 'REGULAR'),
(1, 'B', 1, 'PREMIUM'),
(1, 'B', 2, 'PREMIUM'),
(1, 'B', 3, 'PREMIUM');

INSERT INTO shows
(screen_id, movie_id, show_date, start_time, end_time, base_price)
VALUES
(1, 1, '2026-09-28', '10:00:00', '12:25:00', 220.00),
(1, 1, '2026-09-28', '14:00:00', '16:25:00', 250.00),
(1, 1, '2026-09-28', '19:00:00', '21:25:00', 300.00);

INSERT INTO show_seats (show_id, seat_id, price)
SELECT sh.show_id, s.seat_id,
       CASE
           WHEN s.seat_type = 'REGULAR' THEN sh.base_price
           WHEN s.seat_type = 'PREMIUM' THEN sh.base_price + 50
           ELSE sh.base_price + 150
       END
FROM shows sh
JOIN seats s ON s.screen_id = sh.screen_id;
```

Normalization

1NF: All columns contain atomic values. Seats and other repeated values are stored as separate rows.

2NF: Non-key attributes depend on the complete key. This is followed in the composite-key table booking_seats.

3NF: Movie, theatre, screen and seat information is kept in separate tables, so non-key attributes are not unnecessarily repeated.

BCNF: The tables use primary keys and unique constraints for the important functional dependencies, such as theatre_id + screen_name and screen_id + show_date + start_time.

High Concurrency / Locking Strategy

For seat booking, show_seats is the table used to lock a seat for a particular show. I will use a MySQL transaction with SELECT ... FOR UPDATE so that two users cannot successfully take the same seat at the same time.

```
START TRANSACTION;

SELECT show_seat_id, seat_id, status, hold_expires_at
FROM show_seats
WHERE show_id = 1
  AND seat_id IN (1, 2)
ORDER BY seat_id
FOR UPDATE;

-- Check that the seats are AVAILABLE.

UPDATE show_seats
SET status = 'HELD',
    hold_token = UUID(),
    hold_expires_at = DATE_ADD(NOW(), INTERVAL 5 MINUTE),
    version = version + 1
WHERE show_id = 1
  AND seat_id IN (1, 2)
  AND status = 'AVAILABLE';

COMMIT;
```

The unique key on show_seats(show_id, seat_id) makes sure that only one inventory row exists for a seat in a particular show. The row lock prevents concurrent users from changing the same seat at the same time.

P2 – Query

List all shows on a given date at a given theatre, along with their show timings:

```
SELECT
    sh.show_id,
    m.title AS movie_name,
    sc.screen_name,
    sh.show_date,
    sh.start_time,
    sh.end_time,
    sh.base_price
FROM shows sh
JOIN screens sc
    ON sc.screen_id = sh.screen_id
JOIN movies m
    ON m.movie_id = sh.movie_id
WHERE sc.theatre_id = ?
  AND sh.show_date = ?
  AND sh.status = 'SCHEDULED'
ORDER BY sh.start_time;
```

Payment Webhook Idempotency

```
INSERT INTO payment_webhooks
(provider, provider_event_id, event_type, provider_payment_id, payload)
VALUES
('RAZORPAY', 'event_10001', 'payment.captured',
 'pay_10001', '{"amount":25000,"currency":"INR"}')
ON DUPLICATE KEY UPDATE
webhook_id = webhook_id;
```